

Theory:

Ultrasound imaging is a medical imaging technique that uses high-frequency sound waves to visualize internal structures of the body. These sound waves are mechanical waves that travel through tissues by causing particles to vibrate in a longitudinal manner. The frequency used in medical ultrasound is typically above 1 MHz, which is much higher than the audible range for humans. Its working principle is based on pulse-echo technique. A device called transducer contains piezoelectric crystal. When it vibrates, it produces waves from electrical energy. These waves travel through the body and encounter different tissues. When the ultrasound waves reach boundaries between different types of tissues, known as acoustic interfaces, a portion of the wave is reflected back toward the transducer while the rest continues deeper into the body. These reflected waves are called echoes. The transducer then acts as a receiver and converts these returning echoes into electrical signals.

Feature Extraction

```
I = imread(...)
```

Reads the original ultrasound image.

```
I = rgb2gray(I);
```

Converts the image to grayscale.

```
I = im2double(I);
```

Normalizes the image.

```
I3 = I .* I2;
```

Masking operation, keeps only the lesion region and removes the background.

```
cc = bwconncomp(I2);
```

This function identifies connected regions in the binary mask. It helps locate the lesion area and prepares the image for further analysis.

```
props = regionprops(cc, 'BoundingBox', 'Area', 'ConvexHull', 'ConvexArea');
```

This function extracts:

Area → number of pixels in lesion

BoundingBox → rectangular boundary around lesion

ConvexHull → smooth outer boundary

```
lesion_area = props.Area / numel(I);
```

This measures how large the tumor is relative to the image.

```
lesion_aspect_ratio = height / width;
```

Obtained from bounding box dimensions. It describes the shape of the lesion.

```
mean = sum(pixelValues)/length(pixelValues);
```

Represents average brightness of lesion.

```
std = std(pixelValues);
```

Represents variation in intensity (texture).

```
boundaries = bwboundaries(I2);
```

Extracts the edge of the lesion.

```
[k,v] = convhull(...)
```

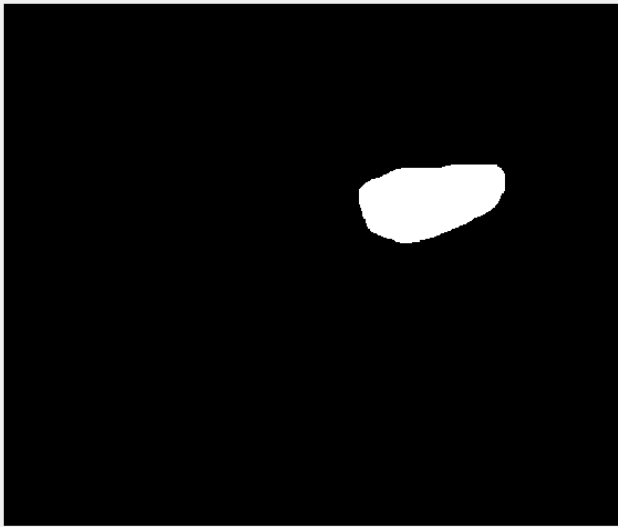
Used to compute a smooth boundary around lesion.

Results

```
%%  
clear all  
close all  
clc  
I = imread(['benign/benign (3).png']);  
I = rgb2gray(I);  
I = im2double(I);  
imshow (I)
```



```
%%  
I2 = imread('benign/benign (3)_mask.png');  
figure(2)  
imshow(I2)  
impixelinfo()
```



Pixel info: (X, Y) BW

```
%%  
I3= I.*I2;  
figure(3)  
imshow(I3)
```



```
%%  
cc = bwconncomp(I2);  
props = regionprops(cc, 'BoundingBox', 'Area', 'ConvexHull', 'ConvexArea');  
props.Area
```

```
ans = 7022
```

```
%%  
[row,col]=find(I3>0 & I3<.8); %Selecting this range to avoid markings/writings  
inside lesion  
pixelValues = I3(sub2ind(size(I3), row, col));  
mean = sum(pixelValues)/length(pixelValues)
```

```
mean = 0.2442
```

```
standard_deviation = std(pixelValues)
```

```
standard_deviation = 0.1155
```

```
%%  
boundaries = bwboundaries(I2);  
boundary = boundaries{1};  
[k, v] = convhull(boundary(:,2), boundary(:,1));  
perimeterOriginal = sum(sqrt(sum(diff(boundary([1:end,1],:)).^2,2))); % Finding  
the length of the original  
boundaryHull = boundary(k, :); % Points on the convex hull  
perimeterConvexHull = sum(sqrt(sum(diff(boundaryHull).^2, 2))); % Finding the  
length of convex boundary  
irregularityIndex = perimeterOriginal / perimeterConvexHull
```

```
irregularityIndex = 1.0563
```

```
%%  
clc; clear;  
patient = {};
```

```

for i=1:1:210    % malignant 210, benign 437
    fileName = strcat('malignant/malignant ',num2str(i),'.png');    % malignant
/ benign
    I = imread(fileName);
    fileName = strcat('malignant/malignant ',num2str(i),'_mask.png'); %
malignant / benign
    M = imread(fileName); %Masked Image (M)
    I = rgb2gray(I);
    I = im2double(I);
    L= I.*M; %Lesion Segmented Image (L)
    cc = bwconncomp(M); %Finding the boundary of the Binary Masked Image
    props = regionprops(cc, 'BoundingBox', 'Area');
    props=props(1);
    %Feature 1
    lesion_area(i)=props.Area/numel(I);
    %Feature 2
    lesion_aspect_ratio(i) = props.BoundingBox(4)/props.BoundingBox(3); %Depth
(or height) divided by
    boundaries = bwboundaries(M);
    [mx, idx] = max(arrayfun(@(x) size(cc.PixelIdxList{x}, 1) ,
1:size(cc.PixelIdxList, 2))));
    boundary = boundaries{idx}; % choose largest boundary
    try
        [k, v] = convhull(boundary(:,2), boundary(:,1));
        perimeterOriginal = sum(sqrt(sum(diff(boundary([1:end,1],:)).^2,2)));
        boundaryHull = boundary(k, :); % Points on the convex hull
        perimeterConvexHull = sum(sqrt(sum(diff(boundaryHull).^2, 2)));
        %Feature 3
        irregularityIndex(i) = perimeterOriginal / perimeterConvexHull;
    catch
        disp('caught');
        disp(i);
        irregularityIndex(i) = mean(irregularityIndex);
    end
    %Quantitative Features - Depend on the pixel values of the lesion
    [row,col]=find(L>0 & L<.8);
    pixelValues = L(sub2ind(size(L), row, col));
    %Feature 4
    lesion_mean(i) = sum(pixelValues)/length(pixelValues);
    %Feature 5
    lesion_std(i) = std(pixelValues);

    patient{i} = cellstr(strcat('M_',num2str(i)));    % M_ / B_
    target_variable{i} = '1';                        % benign 0    malignant 1
end
patient =string(patient);
target_variable=string(target_variable);

```

```

feature_set =
table(patient',lesion_area',lesion_aspect_ratio',irregularityIndex',lesion_mean',
lesion_std',target_variable', ...

'VariableNames',["Patient","lesion_area","lesion_aspect_ratio","irregularityIndex",
"lesion_mean","lesion_std","target_variable"]);

% Specify the filename
filename = 'feature_set_malignant.xlsx'; % benign malignant
% Write the table to an Excel file
writetable(feature_set, filename);

```

This section will generate two excel files containing the benign and malignant cases.

Accuracy Discussion

SVM shows high accuracy due to clear separation.

Random Forest is robust and handles feature variation well.

Decision Tree may overfit.

Logistic Regression works but less powerful for complex patterns.

Linear Regression

fit_time	0.012010
score_time	0.016569
test_accuracy	0.944423
test_recall	0.871429
test_specificity	0.979493
test_gmean	0.922779
test_roc	0.925461
test_mcc	0.874236
dtype: float64	

SVM

•• fit_time	0.018630
score_time	0.017648
test_accuracy	0.941274
test_recall	0.852381
test_specificity	0.983985
test_gmean	0.914709
test_roc	0.918183
test_mcc	0.866787
dtype: float64	

DT

```
.. fit_time          0.010203
   score_time        0.017804
   test_accuracy      0.969135
   test_recall        0.952381
   test_specificity    0.977220
   test_gmean         0.964178
   test_roc           0.964800
   test_mcc           0.931590
   dtypes: float64
```

Discussion:

A good masking technique is important as all the extracted features depend directly on the segmented region. If the mask does not represent the segmented lesion correctly, the background pixels may be included or parts of the tumors may be missed. This will lead to incorrect feature values.

Deep learning models automatically provide segmentation, so manual masking is not needed. A high-quality deep learning segmentation model can accurately identify the tumor boundaries even in complex ultrasound images. However, if the model is poorly trained or the dataset is insufficient, it may produce incorrect boundaries.

Low resolution ultrasound images can deteriorate the classification task. Ultrasound images already suffer from noise and low contrast, reducing the resolution will worsen the situation further. Important details like edges, texture variations and small irregularities in tumor boundaries become blurred or lost. This will make a difficult task to separate the benign and malignant cases. Low resolution can affect segmentation itself. If the tumor boundary is not clearly visible, the masking process becomes inaccurate. This introduces errors in all subsequent steps.